

Quick Start Guide - GA VRPTW Optimization

This guide walks you through using the three different interfaces to run GA VRPTW experiments.

Installation

```
# Install dependencies
pip install matplotlib numpy
```

Ensure you're in the project directory:

```
cd /path/to/genetic_algorithm
```

Method 1: Basic UI (Recommended for Users)

The easiest way to run experiments with a graphical interface.

Start the application:

```
python run_ui.py
```

Steps:

1. Configuration Tab

- Click "📁 Browse Data Folder" to select instances (e.g., C101.txt, R102.txt)
- You can select multiple files
- Check the operators you want to use:
 - Initialization: Choose at least one (Random, Solomon, etc.)
 - Crossover: Choose at least one (Edge Assembly, OX, etc.)
 - Mutation: Choose at least one (2-Opt, Or-Opt, etc.)
 - Selection: Choose one (default is Tournament)
- Set parameters:
 - Population Size: 20-50 for quick tests, 50-100 for thorough
 - Generations: 50-100 for testing, 200+ for detailed analysis
 - Mutation Rate: 0.1-0.3 typical
 - Number of Runs: 1-3 for testing, 5+ for statistics
- Click "▶ Run Experiment"

2. Results Tab

- Select an instance to view
- Click " Plot Selected" to see:
 - Convergence curves (fitness over generations)
 - Fitness distribution (box plot of best fitnesses)
 - Statistical summary

Example Workflow:

```
1. Select: C101.txt, C102.txt
2. Choose: Random init, Edge Assembly crossover, 2-Opt mutation
3. Set: Population=50, Generations=100, Runs=3
4. Wait for completion (~2-5 minutes)
5. View results in Results tab
```

Method 2: Advanced UI (For Comparisons)

Includes operator comparison tools for analyzing performance.

Start the advanced application:

```
python run_advanced_ui.py
```

Features:

- All features from Basic UI
- **Operator Comparison Tab:** Compare performance across instances
 - Select multiple instances to compare side-by-side
 - Click " Compare Selected" to generate comparison plots
 - Click " Compare All" for overview across all instances

Example Workflow:

```
1. Run basic experiments with different operators
2. Switch to Operator Comparison tab
3. Select C101, C102, R101
4. Click "Compare Selected" to see side-by-side plots
```

Method 3: Command-Line Interface (For Automation)

Best for batch experiments and scripting.

Basic usage:

```
python ga_cli.py run C101.txt
```

With custom parameters:

```
python ga_cli.py run C101.txt \  
  --population 100 \  
  --generations 200 \  
  --mut-rate 0.2 \  
  --runs 5 \  
  --init solomon \  
  --crossover edge \  
  --mutation 2opt \  
  --selection tournament
```

List available options:

```
python ga_cli.py list instances    # Show all data files  
python ga_cli.py list operators   # Show all operators
```

Batch script example (run_batch.sh):

```
#!/bin/bash  
python ga_cli.py run C101.txt --init random --crossover pmx --mutation  
oropt --runs 3  
python ga_cli.py run C102.txt --init solomon --crossover edge --mutation  
2opt --runs 3  
python ga_cli.py run R101.txt --init cluster --crossover ox --mutation  
scramble --runs 3
```

Then run:

```
bash run_batch.sh
```

Operator Guide

Initialization Functions

Name	Description	Best For
------	-------------	----------

Name	Description	Best For
Random	Greedy random insertion	Quick solutions
Solomon	Nearest neighbor + spatial	Good quality solutions
Cluster First	Cluster then route	Large problems
Savings	Clarke-Wright savings	Balanced approach
Mixed	Randomly combines all	Diversity in population

Crossover Operators

Name	Description	Best For
PMX	Partially Mapped	TSP-style problems
Order (OX)	Order preserving	Sequential problems
Cycle (CX)	Cycle preservation	Complex tours
Edge Assembly	Route-based (VRPTW optimized)	Time windows
Route-Based	Route-level operations	Vehicle routing

Mutation Operators

Name	Description	Impact
Or-Opt	Move segments	Local improvements
Scramble	Shuffle subsequences	Moderate disruption
2-Opt	Swap edges (local search)	Strong local optimization
Insert	Random insertion	Explores neighbors

PROF

Parameter Recommendations

For Quick Testing (5-10 minutes):

Population: 20
Generations: 50
Mutation Rate: 0.2
Runs: 1

For Moderate Analysis (15-30 minutes):

```
Population: 50
Generations: 100
Mutation Rate: 0.2
Runs: 3
```

For Thorough Comparison (1+ hours):

```
Population: 100
Generations: 200
Mutation Rate: 0.2
Runs: 5
```

Understanding Results

In the Plots Tab:

1. Convergence Curve

- X-axis: Generation number
- Y-axis: Fitness value
- Lines showing "best" (solid) = best solution per generation
- Lines showing "avg" (dashed) = average population fitness
- Better convergence = steady downward trend

2. Fitness Distribution

- Box plot shows median, quartiles, outliers
- Red line = median
- Box = middle 50% of runs
- Points = individual runs
- Tighter box = more consistent operator

3. Statistics Panel

- Best/Worst/Average: Overall performance
- Std Dev: Consistency (lower = more stable)

Troubleshooting

UI doesn't start:

```
# Check Python version
python --version # Should be 3.7+
```

```
# Verify matplotlib is installed
python -c "import matplotlib; print('OK')"
```

"Instance file not found":

- Ensure data files are in the **data/** folder
- Use full path with "Add Custom File" if elsewhere

No results showing:

- Check **results/** folder exists
- Ensure experiments completed successfully
- Try "Refresh" in Results tab

Slow performance:

- Reduce population size
- Reduce number of generations
- Use fewer runs initially
- Try smaller instances (C101 vs C108)

File Structure

```
genetic_algorithm/
├─ run_ui.py                # Start basic UI
├─ run_advanced_ui.py      # Start advanced UI
├─ ga_cli.py               # Command-line tool
├─ data/                   # Benchmark instances
│  └─ C101.txt
│  └─ R101.txt
│  └─ ...
├─ results/                # Experiment results
│  └─ C101/
│     └─ run_001.json
│     └─ summary.json
│  └─ ...
├─ vrptw/view/             # UI code
│  └─ app.py               # Basic UI
│  └─ advanced_app.py      # Advanced UI
├─ ga/                     # GA implementation
├─ operators/              # Crossover/mutation
└─ README files            # Documentation
```

PROF

Advanced Tips

Running Multiple Experiments:

```
# Create a Python script to loop through instances
for instance in C101 C102 C103:
    python ga_cli.py run ${instance}.txt --runs 5
done
```

Comparing Operator Sets:

```
# Test all crossover + mutation combinations
for cross in pmx ox cx edge route; do
    for mut in oropt scramble 2opt insert; do
        python ga_cli.py run C101.txt \
            --crossover $cross --mutation $mut --runs 2
    done
done
```

Exporting Results:

Results are automatically saved as JSON in `results/` folder.

You can process them with Python:

```
import json
with open('results/C101/run_001.json') as f:
    data = json.load(f)
    print(f"Best fitness: {data['best_fitness']}")
```

Questions?

- Check `UI_README.md` for detailed documentation
- Review the code in `vrptw/view/app.py`
- Look at existing GA implementation in `ga/` folder
- Check `main.py` for more advanced examples

Enjoy optimizing your VRP instances!